

EcoTrack – Personal Carbon Footprint Track

Project Description:

EcoTrack is a Generative AI-powered sustainability platform developed to help individuals monitor, understand, and reduce their personal carbon footprint. The application allows users to record their daily travel, food consumption, and household energy usage while automatically calculating carbon emissions using real-world emission factors from the Carbon Interface API.

The platform is built using **React.js** for the frontend, **Flask (Python)** for the backend, and **Firebase Firestore** for cloud-based data storage. EcoTrack integrates **Groq AI (LLaMA 3.3-70B Versatile)** to generate personalized eco-friendly recommendations based on user activities. Users can visualize their environmental impact through interactive dashboards, animated carbon gauges, heatmaps, and trend charts while tracking progress toward their personal carbon budget.

EcoTrack also includes Firebase Authentication, achievement badges, streak tracking, and sustainability analytics, providing users with an engaging and data-driven approach to promoting environmentally responsible behavior.

Scenarios

Scenario 1

A user logs their daily travel by selecting a transport mode (Car, Bus, Bike, or Train), entering the travel distance, and specifying the number of passengers for carpooling. EcoTrack calculates the travel-related carbon emissions using verified emission factors and stores the results in Firebase Firestore.

Scenario 2

A user records their daily food consumption by selecting a diet type (Meat-heavy, Omnivore, Vegetarian, or Vegan) and indicating whether food was wasted. The application calculates food-related emissions, applies a waste penalty when necessary, and updates the user's daily carbon footprint.

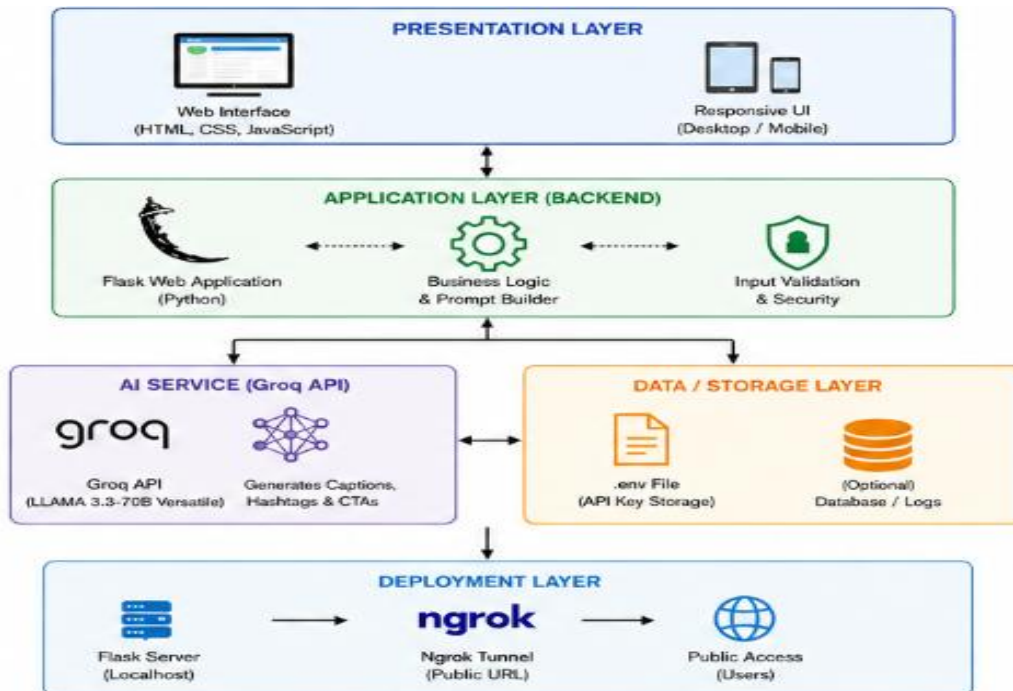
Scenario 3

After logging travel, food, and energy usage, the user opens the AI Suggestions page. EcoTrack sends the daily carbon footprint data to the Groq AI model, which generates personalized sustainability recommendations with estimated carbon savings.

Scenario 4

The user visits the Dashboard to analyze carbon footprint trends. EcoTrack displays interactive charts, carbon budget progress, heatmaps, and achievement badges, helping users monitor long-term sustainability performance.

Technical Architecture:



Description: EcoTrack follows a modular three-tier architecture consisting of a React.js frontend, a Flask backend, Firebase cloud services, and AI integration through the Groq API. The frontend provides a responsive interface for user interaction, while the Flask backend manages authentication, business logic, carbon emission calculations, API communication, and AI prompt processing. Firebase Authentication secures user accounts, and Firebase Firestore stores user profiles, daily activity logs, carbon history, and achievement badges.

The backend communicates with the Carbon Interface API to retrieve verified carbon emission factors and with Groq AI (LLaMA 3.3-70B Versatile) to generate personalized eco-friendly recommendations. Interactive dashboards built using Chart.js visualize user emissions, carbon budgets, historical trends, and sustainability achievements.

(Note: Since the project uses Firebase Firestore as a database, include an ER Diagram along with the Solution Architecture Diagram.)

Project Workflow:

Milestone 1: Model Selection and Architecture

- **Activity 1.1:** Generate a Groq API key and configure it securely in the project environment.
- **Activity 1.2:** Research and select the appropriate generative AI model from Groq for social media content generation.
- **Activity 1.3:** Define the architecture of the application, detailing interactions between the frontend, backend, and AI integration.
- **Activity 1.4:** Set up the development environment, installing necessary libraries and dependencies for Flask and the Groq API.

Milestone 2: Core Functionalities Development

- **Activity 2.1:** Develop the core functionalities: Caption Generator, Hashtag Suggester, and Call-to-Action Generator.
- **Activity 2.2:** Implement the Flask backend to manage routing and user input processing, ensuring smooth API interactions.

Milestone 3: App.py Development

- **Activity 3.1:** Write the main application logic in app.py, establishing routes for each feature and integrating AI responses.

Milestone 4: Frontend Development

- **Activity 4.1:** Design and develop the user interface using HTML, CSS, and JavaScript, ensuring a responsive and intuitive layout.
- **Activity 4.2:** Create dynamic templates with Flask's render_template to display results based on user interactions.

Milestone 5: Deployment

- **Activity 5.1:** Prepare the application for local deployment by configuring the server environment and ensuring all dependencies are correctly installed.
- **Activity 5.2:** Deploy the application publicly using Ngrok to expose the local Flask server over a secure public URL.

Milestone 6: Conclusion

Milestone 1: Model Selection and Architecture

In this milestone, we focus on selecting the appropriate generative AI model from Groq for our social media content generation needs. This involves researching the capabilities and performance of various models that Groq offers, ensuring that the chosen model aligns well with the application's objectives of generating captions, hashtags, and CTAs across different platforms and tones.

Activity 1.1: Generate a Groq API Key

Before the application can communicate with the Groq LLM, you need a valid Groq API key. Follow the steps below to create one and connect it securely to the project.

- **Step 1 — Create a Groq Account:** Visit <https://console.groq.com> and sign up for a free account using your Google account or email address.
- **Step 2 — Navigate to API Keys:** After logging in, click on your profile icon in the top-right corner and select "API Keys" from the dropdown menu, or go directly to the [API Keys page](#).
- **Step 3 — Create a New API Key:** Click the "Create API Key" button. Give your key a descriptive name (e.g., captionai-key) so it is easy to identify later.
- **Step 4 — API Key:** Now give a "Display Name" and "Expiration" is optional, and Click on "Submit". The you will get your API key, Copy it immediately and store it in a safe place you will not be able to view it again after closing the dialog.

- **Step 5 — Add the Key to Your Project:** Create a .env file in the root of your project directory (if it does not already exist) and paste the key as shown below:

```
GROQ_API_KEY=your_copied_api_key_here
```

- **Step 6 — Verify the Key is Loaded:** The app.py file loads this key automatically at startup using python-dotenv:

```
from dotenv import load_dotenv
load_dotenv()
client = Groq(api_key=os.environ.get("GROQ_API_KEY"))
```

- **Important — Never Commit Your API Key:** Add .env to your .gitignore file to prevent accidentally pushing the key to a public repository:
.env

Activity 1.2: Research and Select the Appropriate Generative AI Model

Here are the things that needed to research to select a best model for the project:

- **Understand the Project Requirements:** Review the specific needs of the CaptionAI application, focusing on the types of content it will generate (captions, hashtags, and calls-to-action across Instagram and LinkedIn).
- **Explore Groq's Model Documentation:** Visit the Groq documentation to examine the available LLM models, including their functionalities, context windows, speed, and limitations.

- **Evaluate Model Performance:** Compare models based on response quality, generation speed, and relevance to social media copywriting tasks.
- **Select the Optimal Model:** Choose **llama-3.3-70b-versatile** for its balance of creative output quality and generation speed, set with a temperature of 0.8 and max_tokens of 2048 for rich, varied captions.

Activity 1.3: Define the Architecture of the Application

- **Draft an Architectural Diagram:** Create a visual representation of the application architecture, including components like the frontend (HTML, CSS, JavaScript), Flask backend, and Groq API integration.
- **Detail Frontend Functionality:** Outline how users interact with the application entering product/campaign information, selecting a platform (Instagram or LinkedIn), and choosing a tone (Professional, Casual, or Promotional).
- **Outline Backend Responsibilities:** Specify how the Flask backend handles incoming POST requests to /generate, validates and sanitizes user input, builds a structured prompt, and communicates with the Groq API.
- **Describe AI Integration Points:** Define how the backend constructs prompts using TONE_DESCRIPTIONS and PLATFORM_TIPS dictionaries, calls the Groq API, parses the JSON response via extract_json(), and returns structured results to the frontend.

Activity 1.4: Set Up the Development Environment

- **Install Python and Pip:** Ensure Python 3.8+ is installed on your machine along with pip for dependency management.
- **Create a Virtual Environment:** Set up a virtual environment using venv to isolate project dependencies:

```
D:\projects>python -m venv myenv  
D:\projects>"d:\projects\venv\Scripts\activate"  
(myenv) D:\projects> pip install -r requirements.txt
```

- **Install Flask:** Use pip to install Flask:

```
(myenv) D:\projects> pip install Flask==3.0.3
```

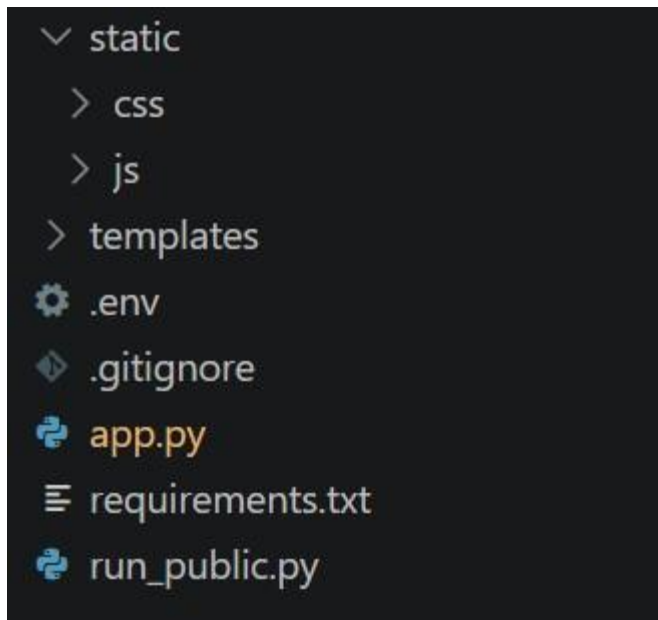
- **Install Groq SDK:** Install the Groq Python library to interact with the Groq API:

```
(myenv) D:\projects> pip install groq==0.11.0
```

- **Install python-dotenv:** Install dotenv support for secure API key management:

```
(myenv) D:\projects> pip install python-dotenv==1.0.1
```

- **Configure the API Key:** Create a .env file in the project root and add your Groq API key:
GROQ_API_KEY=your_groq_api_key_here
- **Set Up Application Structure:** Create the project directory structure including folders for templates, static files (CSS, JS), and main application files (app.py, requirements.txt, run_public.py).



Milestone 2: Core Functionalities Development

Activity 2.1: Develop Core Content Generation Features

Implement the three core generation features Captions, Hashtags, and CTAs as a unified AI response driven by a single structured prompt.

- **Caption Generator:** Generates three unique, platform-appropriate captions. For Instagram/Casual tone, includes relevant emojis and conversational language. For LinkedIn/Professional tone, produces insight-led, value-driven copy.
- **Hashtag Suggester:** Produces ten hashtags per request a curated mix of high-reach popular tags and targeted niche tags relevant to the product or campaign.
- **CTA Generator:** Returns three short (under 10 words), punchy, action-oriented call-to-action phrases calibrated to the selected platform and tone.

Activity 2.2: Implement the Flask Backend

Define Routes in Flask:

- Set up routes in app.py for the home page and content generation endpoint.

Process User Input:

- Create a form in index.html for users to submit their product/campaign information, platform, and tone.
- Use Flask's request.get_json() to retrieve JSON data submitted from the frontend via AJAX.

- Validate that product_info is non-empty and within the 2000-character limit before processing.

Integrate API Calls:

- Within the /generate route handler, call the build_prompt() function to construct a structured prompt combining tone descriptions and platform-specific tips.
- Submit the prompt to the Groq API using client.chat.completions.create() with the llama-3.3-70b-versatile model.
- Parse and validate the JSON response using extract_json(), which handles direct JSON, markdown code blocks, and raw JSON object extraction as fallback strategies.

Milestone 3: App.py Development

Milestone 3 focuses on creating and refining the core logic of the app.py file, which serves as the backbone of the CaptionAI application. In this milestone, you'll set up each feature's functionality through Flask routes, establish reliable prompt construction and API interaction, and conduct unit testing to ensure each feature performs as expected.

Activity 3.1: Writing the Main Application Logic in app.py

Define the Core Routes in app.py:

- Establish routes for CaptionAI's two endpoints, each serving a distinct purpose: ◦ / — Home page route that renders the main index.html interface
- /generate — POST endpoint that accepts JSON input, invokes the AI, and returns structured content

Set Up Route Handlers for Each Feature:

- For the /generate route, implement logic that reads product_info, platform, and tone from the incoming JSON body using request.get_json().
- Apply input validation: return a 400 error if product_info is empty or exceeds 2000 characters.
- Route AJAX requests from the frontend to the /generate endpoint, which processes data and returns a JSON response.

Define Prompt Engineering Helpers:

- **TONE_DESCRIPTIONS** dictionary: maps professional, casual, and promotional tones to detailed copywriting instructions embedded in the prompt.
- **PLATFORM_TIPS** dictionary: maps instagram and linkedin to platform-specific content guidelines (character limits, style expectations).
- **build_prompt()**: assembles a structured prompt string from the product info, selected platform tip, and tone description, instructing the model to return strictly valid JSON.

Integrate the Groq AI Responses in Each Function:

- Call `client.chat.completions.create()` with a system message enforcing JSON-only responses and a user message containing the assembled prompt.
- **extract_json()**: a robust parser that first attempts `json.loads()`, then strips markdown code fences (````json` blocks), and finally uses a regex to locate a raw JSON object ensuring reliable parsing across model response variations.
- Validate that the parsed result contains all three required keys — captions, hashtags, and ctas — before returning it to the frontend.
- **Home route:** `/` → renders `index.html/generate` (POST) → accepts JSON, returns `{success,captions,hashtags,ctas,platform,tone}`

Milestone 4: Frontend Development

Milestone 4 focuses on developing a user-friendly and visually appealing interface for CaptionAI. This involves designing a glassmorphism-style layout with responsive HTML and CSS to enhance usability, and creating dynamic content rendering with vanilla JavaScript to display AI-generated captions, hashtags, and CTAs based on user interactions.

Activity 4.1: Designing and Developing the User Interface

Set Up the Base HTML Structure:

- Develop the main HTML file (index.html) as the single-page interface, including a header with the CaptionAI brand logo and tagline, an input section, and a results section.
- Use semantic HTML elements to keep the structure organized and ensure the interface is accessible for all users.
- Integrate Font Awesome icons for platform indicators and result section headings, and Google Fonts (Inter) for clean typography.

Design a Responsive Layout Using CSS:

- Create a CSS stylesheet (static/css/style.css) implementing a dark glassmorphism aesthetic with animated background blobs (blob-1, blob-2, blob-3) for visual depth.
- Use flexbox and grid layouts to organize the results into a two-column structure (captions on the left, hashtags and CTAs on the right).
- Add media queries to ensure the layout adapts across desktop, tablet, and mobile screen sizes.

Create Separate UI Components for Each Output Type:

- **Caption Cards:** individual cards per caption, each with a one-click copy-to-clipboard button that provides visual feedback (checkmark + green highlight for 2 seconds).
- **Hashtag Chips:** rendered as pill-shaped chip elements inside a glass card, with automatic # prefix validation.
- **CTA List:** displayed as a clean unordered list inside a separate glass card with a bullhorn icon header.

Activity 4.2: Creating Dynamic Content Rendering with JavaScript

Handle Form Submission Asynchronously:

- Attach a submit event listener to the generator form that prevents default form submission and sends the data as a JSON POST request to /generate via the Fetch API.
- During loading, disable the submit button and show an animated CSS loader in place of the button text and arrow icon.

Render Results Dynamically:

- Implement the `renderResults(data)` function in `main.js`, which dynamically creates and injects DOM elements for captions, hashtags, and CTAs into their respective containers.
- After rendering, automatically smooth-scroll the viewport to the results section using `scrollIntoView()`.

Restore UI State After Generation:

- In the finally block of the async handler, re-enable the submit button and restore the original button text and icon regardless of success or failure.

Milestone 5: Deployment

In Milestone 5, the focus is on deploying the CaptionAI application first locally to verify correct operation, and then publicly via Ngrok to share the app over a secure, accessible URL. This involves configuring the server environment, managing dependencies, and launching the app for real-world use.

Activity 5.1: Preparing the Application for Local Deployment

Set Up a Virtual Environment:

- Open a New Terminal
- Then open "Command Prompt"
- Create and activate a virtual environment, then install all required dependencies from requirements.

Configure Environment Variables:

- Create a .env file in the project root and add your Groq API key. The app loads this automatically using python-dotenv at startup: GROQ_API_KEY=your_groq_api_key_here

Activity 5.2: Local Testing and Verification

Start the Flask Development Server:

- Run the application locally using:

```
(myenv) D:\projects>python app.py
```

- Once running, open a browser and navigate to "http://127.0.0.1:5050" to access the app.

```
(venv) D:\projectsSB\AI in healthcare\startup-validator>python app.py
* Serving Flask app 'app'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment.
d. Follow link (ctrl + click)
* Running on http://127.0.0.1:5050
Press CTRL+C to quit
* Restarting with stat
* Debugger is active!
* Debugger PIN: 442-110-195
```

- Interact with the caption generator test all three tones (Professional, Casual, Promotional) and both platforms (Instagram, LinkedIn) to verify the AI responses are correctly generated, parsed, and rendered.

Activity 5.3: Public Deployment via Ngrok

Ngrok creates a secure tunnel from a public URL to your local Flask server, making the app accessible from anywhere without a cloud hosting provider.

Install Ngrok and pyngrok:

- Install the pyngrok Python wrapper, which manages Ngrok programmatically within your Python project:

```
D:\projects>pip install pyngrok
```

- Download and install the Ngrok client from <https://ngrok.com/download>.
- Click on “Microsoft Store Installer”.

Configure Your Ngrok Authtoken:

- Sign up at ngrok.com and copy your authtoken from the Ngrok dashboard.
- The run_public.py script sets the authtoken automatically using:

```
D:\projects>ngrok.set_auth_token("YOUR_NGROK_AUTHTOKEN")
```

- Replace the TOKEN value in run_public.py with your own Ngrok authtoken before running.

Run the Public Deployment Script:

- Instead of running app.py directly, launch run_public.py to start both the Ngrok tunnel and the Flask server:

```
D:\projects>python run_public.py
```

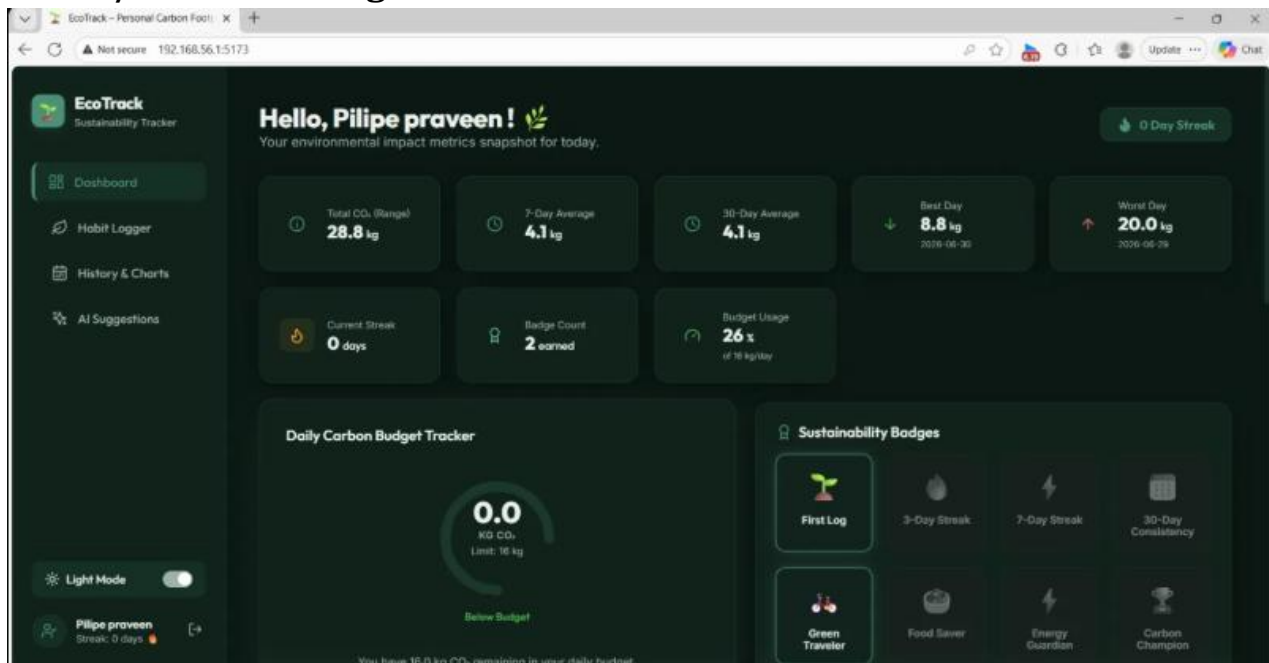
- The script opens an HTTP tunnel on port 5000 and prints the live public URL in the terminal:
===== YOUR
PUBLIC WEBSITE URL IS LIVE!
URL: https://xxxx-xx-xx-xxx-xx.ngrok-free.app
=====
- Share this URL with anyone who can access your CaptionAI app directly from their browser without any installation.

Important Notes:

- **debug=False** is required in run_public.py to prevent Flask from spawning a reloader process, which would interfere with Ngrok’s tunnel management.
- The public URL changes each time you restart run_public.py (on the free Ngrok plan). For a persistent URL, upgrade to a paid Ngrok plan and configure a reserved domain.

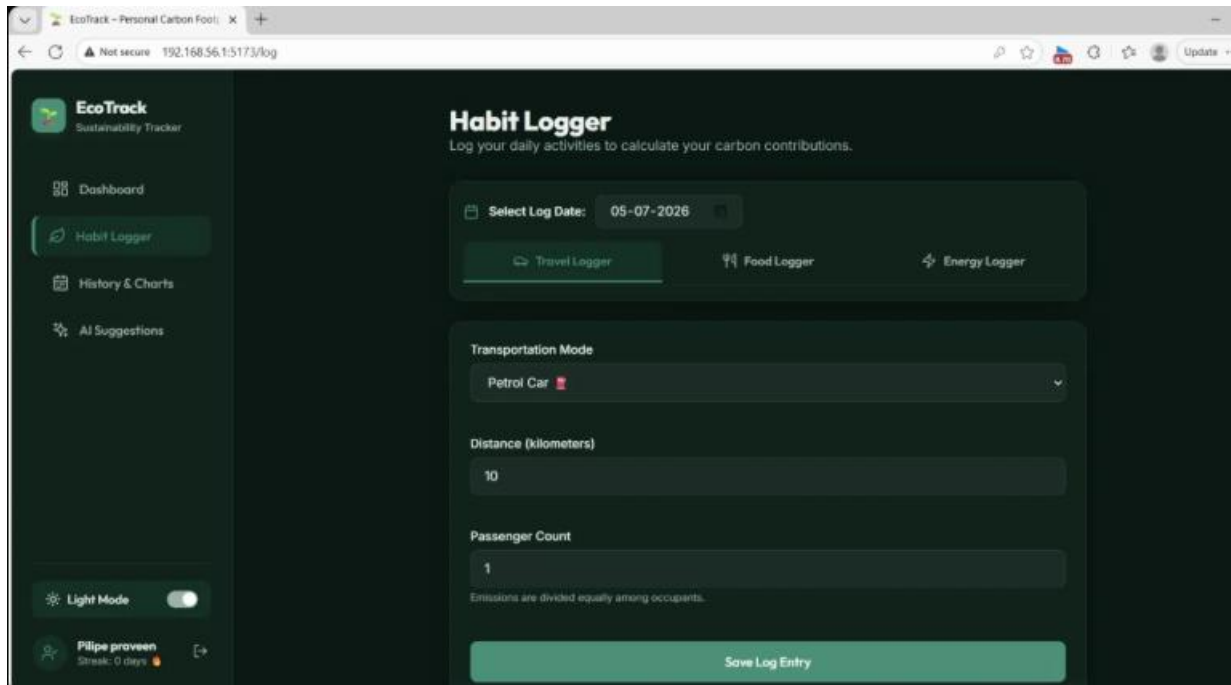
Exploring the Website's Web Pages:

Home / Generator Page:



Description: The single-page interface of CaptionAI serves as both the landing page and the generation tool. It features a dark glassmorphism design with animated background blobs, a header displaying the CaptionAI brand (wand-magic-sparkles icon + logo), and a tagline. The main section contains the input form, platform toggle, tone selector, and the AI-generated results panel all on one seamless page.

Input Section:



EcoTrack
Sustainability Tracker

- Dashboard
- Habit Logger**
- History & Charts
- AI Suggestions

Light Mode ☐

Philippe proven
Streak: 0 days

Habit Logger

Log your daily activities to calculate your carbon contributions.

Select Log Date: 05-07-2026

Travel Logger | Food Logger | Energy Logger

Transportation Mode
Petrol Car

Distance (kilometers)
10

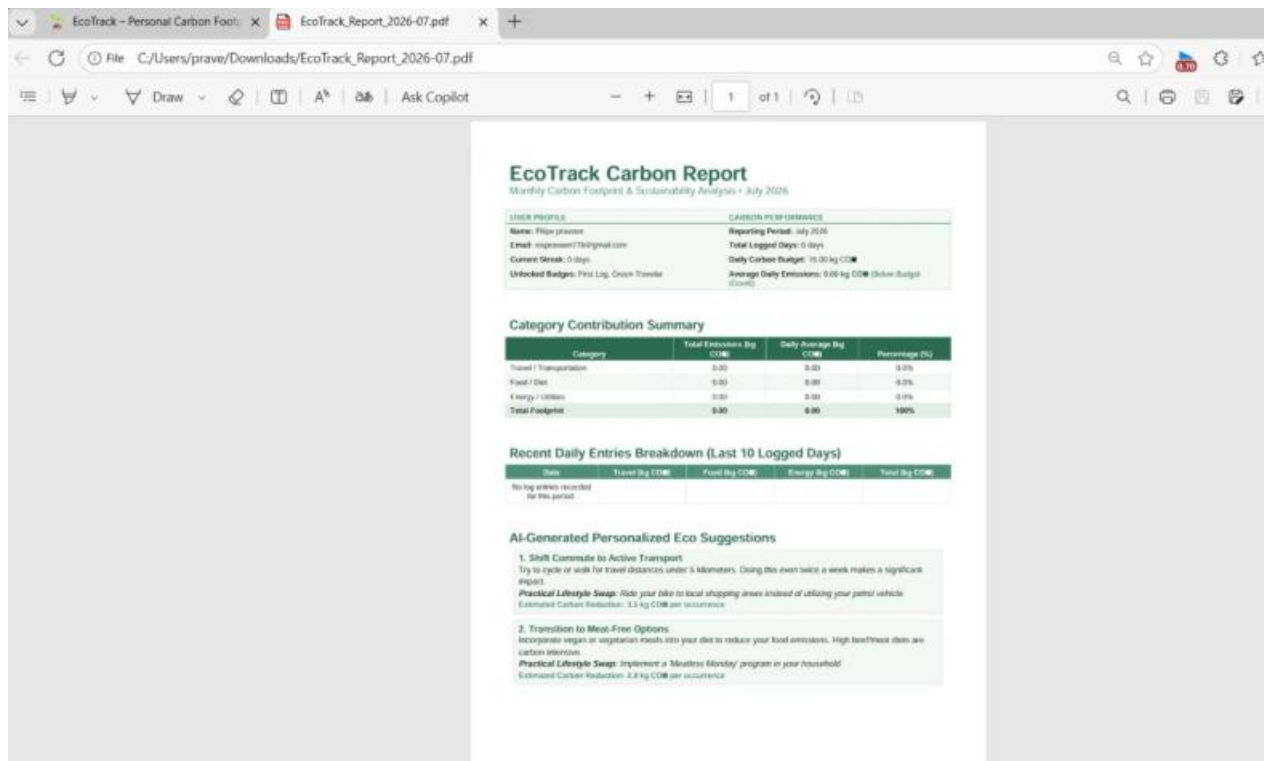
Passenger Count
1

Emissions are divided equally among occupants.

Save Log Entry

Description: A glass-styled card containing the main generation form. Users enter their product, service, or campaign description in a textarea (up to 2000 characters), select their target platform via a toggle button group (Instagram or LinkedIn), choose their tone of voice (Professional, Casual, or Promotional), and click the Generate Content button. While the AI processes the request, the button transitions to an animated loading spinner.

Results Section:



EcoTrack Carbon Report

Monthly Carbon Footprint & Sustainability Analysis • July 2026

USER PROFILE	CARBON & ECO DATA
Name: Philip proven	Reporting Period: July 2026
Email: philip@smartbridge.com	Total Logged Days: 0 days
Current Streak: 0 days	Daily Carbon Budget: 10.00 kg CO ₂ e
Unlocked Badges: First Log, Green Traveler	Average Daily Emissions: 0.00 kg CO ₂ e (Below Budget)

Category Contribution Summary

Category	Total Emissions (kg CO ₂ e)	Daily Average (kg CO ₂ e)	Percentage (%)
Travel / Transportation	0.00	0.00	0.0%
Food / Diet	0.00	0.00	0.0%
Energy / Utilities	0.00	0.00	0.0%
Total Footprint	0.00	0.00	100%

Recent Daily Entries Breakdown (Last 10 Logged Days)

Date	Travel (kg CO ₂ e)	Food (kg CO ₂ e)	Energy (kg CO ₂ e)	Total (kg CO ₂ e)
No log entries recorded for this period.				

AI-Generated Personalized Eco Suggestions

- Shift Commute to Active Transport**
Try to cycle or walk for travel distances under 5 kilometers. Doing this even twice a week makes a significant impact.
Practical Lifestyle Swap: Ride your bike to local shopping areas instead of utilizing your petrol vehicle.
Estimated Carbon Reduction: 3.5 kg CO₂e per occurrence.
- Transition to Meat-Free Options**
Incorporate vegan or vegetarian meals into your diet to reduce your food emissions. High beef/pork diets are carbon intensive.
Practical Lifestyle Swap: Implement a 'Meatless Monday' program in your household.
Estimated Carbon Reduction: 2.8 kg CO₂e per occurrence.

Description: Revealed dynamically after generation, this section displays the AI output in a two-column grid. The left column shows three caption cards, each with the generated caption text and a copy-to-clipboard button. The right column contains two glass cards: one with ten hashtag chips (prefixed with #) and one with three CTA phrases in a list format. The viewport automatically smooth-scrolls to this section upon content generation.

Conclusion:

CaptionAI is a generative AI platform built with Flask and styled with a glassmorphism UI that streamlines social media content creation. It uses Groq's high-speed inference API with the LLaMA 3.3-70B Versatile model to instantly generate tailored captions, hashtags, and calls-to-action for Instagram and LinkedIn across professional, casual, and promotional tones. The project, which included model selection, development, and deployment via Ngrok, highlights how targeted AI and a structured framework can boost productivity for marketers and creators, with potential for future scalability. Looking ahead, CaptionAI offers significant opportunities for expansion, such as integrating multi-language support, adding image analysis capabilities for visual-context generation, and implementing user authentication to save generation history. By continuously refining the underlying model prompts and enhancing the frontend accessibility, the platform is well-positioned to evolve from a specialized tool into a comprehensive social media management assistant for teams of all sizes.